DATA STRUCTURES (ETCCDS202)

Algorithmic Efficiency & Recursion Toolkit (AERT)

Name: Samarth Sharma

Roll No: 2501730279

Course: B.Tech CSE(AI ML), 2nd Sem

Unit: Foundations & Algorithmic Analysis

1. Introduction

In this assignment, I implemented a small toolkit called AERT (Algorithmic Efficiency & Recursion Toolkit). The goal of this project was to understand recursion, algorithm analysis, ADT concepts, and paradigms like Divide & Conquer and Dynamic Programming.

---

2. Stack ADT

What is an ADT?

An Abstract Data Type (ADT) defines what operations can be performed but does not specify how they are implemented internally.

For example, a Stack ADT allows:

- push
- pop
- peek
- is_empty
- size

But it does not specify whether we use arrays or linked lists internally.

Why Stack operations are O(1)?

Stack operations like push and pop operate only at the top of the stack. They do not require traversing the entire structure. Therefore, time complexity is constant:

Time Complexity = **O(1)**

---

3. Factorial (Recursive)

Algorithm Idea

Factorial is defined as:

$n! = n \times (n-1)!$

Base Case:
$0! = 1$

Time Complexity

Each call reduces n by 1.

$T(n) = T(n-1) + O(1)$

Time Complexity:

- Big-O: $O(n)$
- Big-Theta: $\Theta(n)$
- Big-Omega: $\Omega(n)$

Space Complexity

Recursion depth = n
So space complexity = $O(n)$

---

4. Fibonacci

Naive Fibonacci

Recursive formula:

$F(n) = F(n-1) + F(n-2)$

Time Complexity

$T(n) = T(n-1) + T(n-2) + O(1)$

Time Complexity:

- Big-O: $O(2^n)$
- Very slow for large n

Why it becomes slow?

Because it recalculates the same values multiple times.

Example:
F(5) computes F(3) multiple times.

Memoized Fibonacci (Dynamic Programming)

Memoization stores already computed results.

Time Complexity:

- Big-O: $O(n)$
- Big-Theta: $\Theta(n)$
- Big-Omega: $\Omega(n)$

Space Complexity:

- $O(n)$ for recursion stack
- $O(n)$ for memo table

This version is much faster.

---

5. Tower of Hanoi

Recursion Idea

To move n disks:

1. Move n-1 disks to auxiliary
2. Move nth disk
3. Move n-1 disks to destination

Base Case:
If $n = 1 \rightarrow$ directly move disk

Why moves = $2^n - 1$?

Recurrence:

$T(n) = 2T(n-1) + 1$

Solving gives:

$T(n) = 2^n - 1$

Complexity

Time Complexity:
$O(2^n)$

Space Complexity:
O(n) (recursion depth)

---

6. Recursive Binary Search

Binary search works only on sorted arrays.

Recurrence

$T(n) = T(n/2) + O(1)$

Time Complexity:
$O(\log n)$

Cases

Best Case:
Key is middle element → $O(1)$

Worst Case:
Key found at last level or not present → $O(\log n)$

Average Case:
$O(\log n)$

---

7. Paradigm Reflection

Binary Search → Divide & Conquer

Because:

- Problem is divided into two halves
- Only one half is solved recursively

Fibonacci Memoized → Dynamic Programming

Because:

- It solves overlapping subproblems
- Stores results to avoid recomputation

Real-life example:
Saving calculated marks instead of recalculating total again and again.

Tower of Hanoi → Recursion

Because:

- Solves smaller subproblem (n-1)
- Has a clear base case
- Follows self-similar structure

Real-Life Examples

Divide & Conquer:
Searching a word in dictionary by opening middle pages.

Dynamic Programming:
Saving previously solved math problems for revision instead of solving again.

Greedy:
Giving change using highest denomination first.

---

8. Conclusion

Through this assignment, I understood recursion deeply, especially how base cases and call stack work. I also learned how time complexity differs significantly between naive recursion and optimized solutions like memoization. This assignment helped me connect theory of Big-O with practical coding.